Mitigating AI Hallucinations, Fabricated APIs & Over-Reliance

Question: While integrating a newly released third-party SDK, GitHub Copilot suggests a method call `client.verify_webhook_signature_v2()` that looks idiomatic and passes static syntax highlighting, but throws an `AttributeError` at runtime because the method does not exist in the library. What is the primary cause of this behavior, and what engineering practice best mitigates it?

A. A corrupted local VS Code extension cache; reinstalling Copilot eliminates non-existent method suggestions permanently.

B. Probabilistic token generation (LLM hallucination) predicting plausible API names; mitigated by human code review, automated unit/integration tests, and grounding prompts with actual SDK source/docs. **[CORRECT ANSWER]**

C. The Public Code Matching filter automatically rewrites real SDK methods into fictional names to avoid copyright infringement.

D. Enabling strict Content Exclusions on `node_modules` or `site-packages` prevents Copilot from inventing package methods.

Explanation & Exam Rationale

Because generative language models predict the most statistically probable sequence of tokens rather than executing a compiler or verifying live package registries, they can produce **hallucinations**—plausible-sounding but non-existent functions, flags, or CVE patches. Responsible AI usage requires treating Copilot as a pair programmer rather than an infallible authority.

Risk Category	Manifestation in Code	Recommended Safeguard
Fabricated Packages / APIs	Non-existent methods, parameters, or unmaintained package names	Verify imports against official docs/lockfiles; run automated test suites
Insecure Code Patterns	Hardcoded credentials, weak hashes (MD5/SHA1), or SQL concatenation	Run static analysis (CodeQL / SAST) and secret scanning in CI/CD
Automation Bias	Accepting large multi-line completions blindly without reading	Enforce mandatory peer code review and incremental testing

Key Exam Takeaway: On the GH-300 exam, the right mitigation for Copilot hallucinations or insecure suggestions is always a combination of **human verification + automated testing + static security scanning (CI/CD)**, supplemented by grounding context (`#file` or Knowledge Bases).